



9

UNDERSTANDING OPENGL



In this project, you'll create a simple program that displays a texture-mapped square using OpenGL and GLFW. OpenGL is a software interface to your graphics processing unit (GPU), and GLFW is a windowing toolkit. You'll also learn how to use the C-like OpenGL Shading Language (GLSL) to write *shaders*—code that executes in the GPU. Shaders bring immense flexibility to computations in OpenGL. I'll show you how to use GLSL shaders to transform and color geometry as you create a rotating, textured polygon (as shown in **Figure 9-1**).

GPUs are optimized to perform the same operations on huge amounts of data repeatedly, in parallel, which makes them much faster than central processing units (CPUs) for rendering computer graphics. In addition, they're being used for general-purpose computing, and specialized languages now let you harness your GPU hardware for all sorts of applications. You'll leverage the GPU, OpenGL, and shaders in this project.

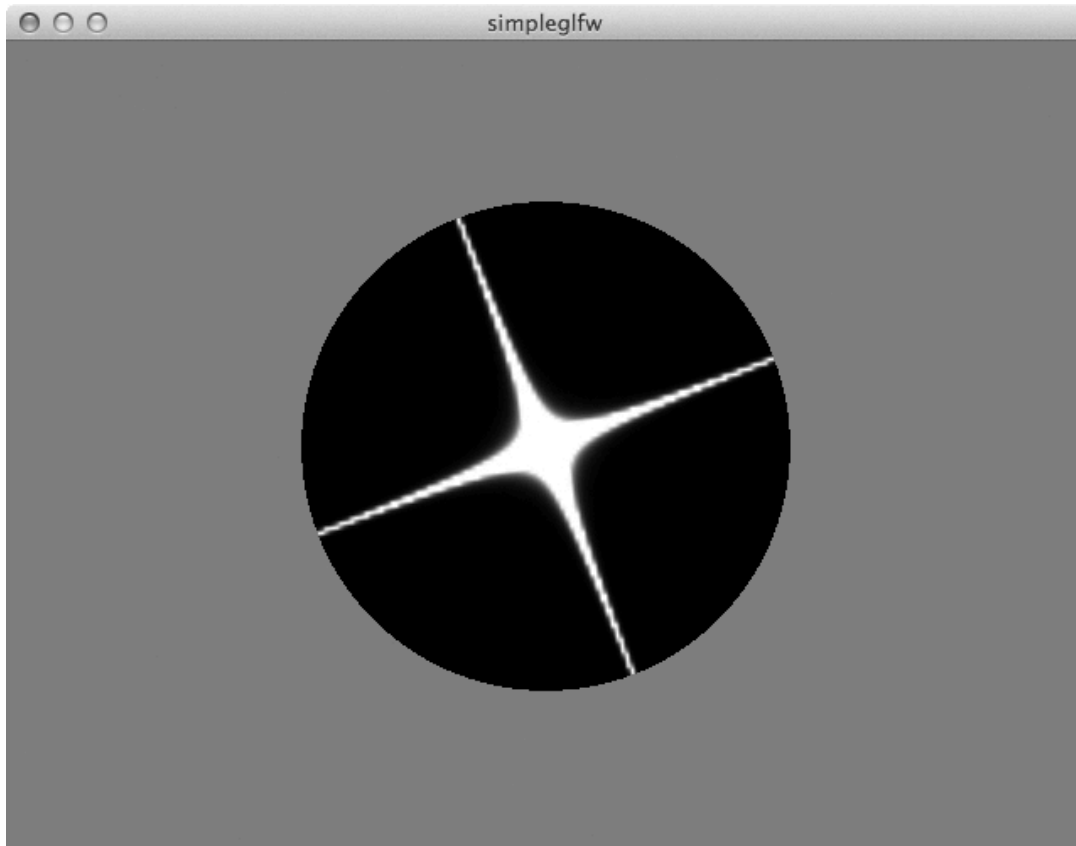


Figure 9-1: The final image for the project in this chapter—a rotating polygon with a star image. This square polygon boundary is clipped to a black circle using a shader.

Python is an excellent “glue” language. There are a vast number of Python *bindings* available for libraries written in other languages, such as C, that allow you to use these libraries in Python. In this chapter and in [Chapters 10](#) and [11](#), you’ll use `PyOpenGL`, the Python binding to OpenGL, to create computer graphics.

Here are some of the concepts introduced in this project:

- Using the GLFW windowing library for OpenGL
- Using GLSL to write vertex and fragment shaders
- Performing texture mapping
- Using 3D transformations

First, let’s take a look at how OpenGL works.

NOTE *OpenGL went through a major transition a few years ago. It went from using a fixed function graphics pipeline to a programmable pipeline with a dedicated shading language. We refer to the latter as modern OpenGL, and that's what we'll be using in this book. Specifically, we'll use OpenGL version 4.1.*

How OpenGL Works

Modern OpenGL makes graphics appear on your screen through a sequence of operations commonly known as the *3D graphics pipeline*.

Figure 9-2 shows a simplified representation of the OpenGL 3D graphics pipeline.

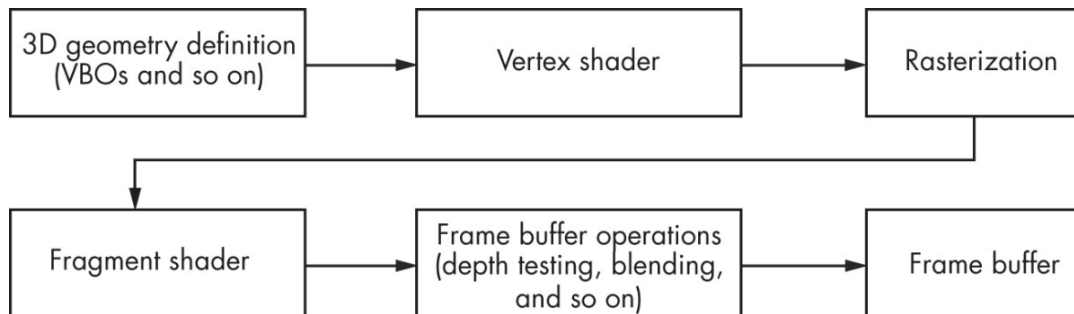


Figure 9-2: *The (simplified) OpenGL graphics pipeline*

At their heart, computer graphics boil down to computing color values for the pixels on your screen. Say you want to make a triangle appear. In the first step of the pipeline, you define the 3D geometry by defining the vertices of the triangle in 3D space and specifying the colors associated with each vertex. These vertices and colors are held in data structures called *vertex buffer objects (VBOs)*. Next, you transform the vertices: the first transformation places the vertices in 3D space, and the second projects the 3D coordinates onto 2D space for display on a 2D screen. The color values for the corresponding vertices are also calculated in this step based on factors such as lighting, typically in code called the *vertex shader*.

Next, the geometry is *rasterized* (converted from a 3D representation to 2D pixels), and for each pixel (or *fragment*, to be more accurate), another block of code called the *fragment shader* is executed. Just as the vertex shader operates on 3D vertices, the fragment shader operates on the 2D fragments after rasterization. I say *fragment* rather than *pixel* since a pixel is what is displayed on the screen, whereas a fragment is the output of computations in the fragment shader, and depending on the next step in the pipeline, a fragment may be discarded before it becomes a pixel on the screen.

Finally, each fragment passes through a series of frame buffer operations, where it undergoes *depth buffer testing* (checking whether one fragment obscures another), *blending* (mixing two fragments with transparency), and other operations that combine its current color with what is already on the frame buffer at that location. These changes end up on the final frame buffer, which is typically displayed on the screen.

Geometric Primitives

Because OpenGL is a low-level graphics library, you can't ask it directly to draw a cube or a sphere, though libraries built on top of it can do such tasks for you. OpenGL understands only low-level geometric primitives, such as points, lines, and triangles.

Modern OpenGL supports only the primitive types `GL_POINTS`, `GL_LINES`, `GL_LINE_STRIP`, `GL_LINE_LOOP`, `GL_TRIANGLES`, `GL_TRIANGLE_STRIP`, and `GL_TRIANGLE_FAN`. **Figure 9-3** shows how the vertices for the primitives are organized. Each vertex has a 3D coordinate such as (x, y, z) .

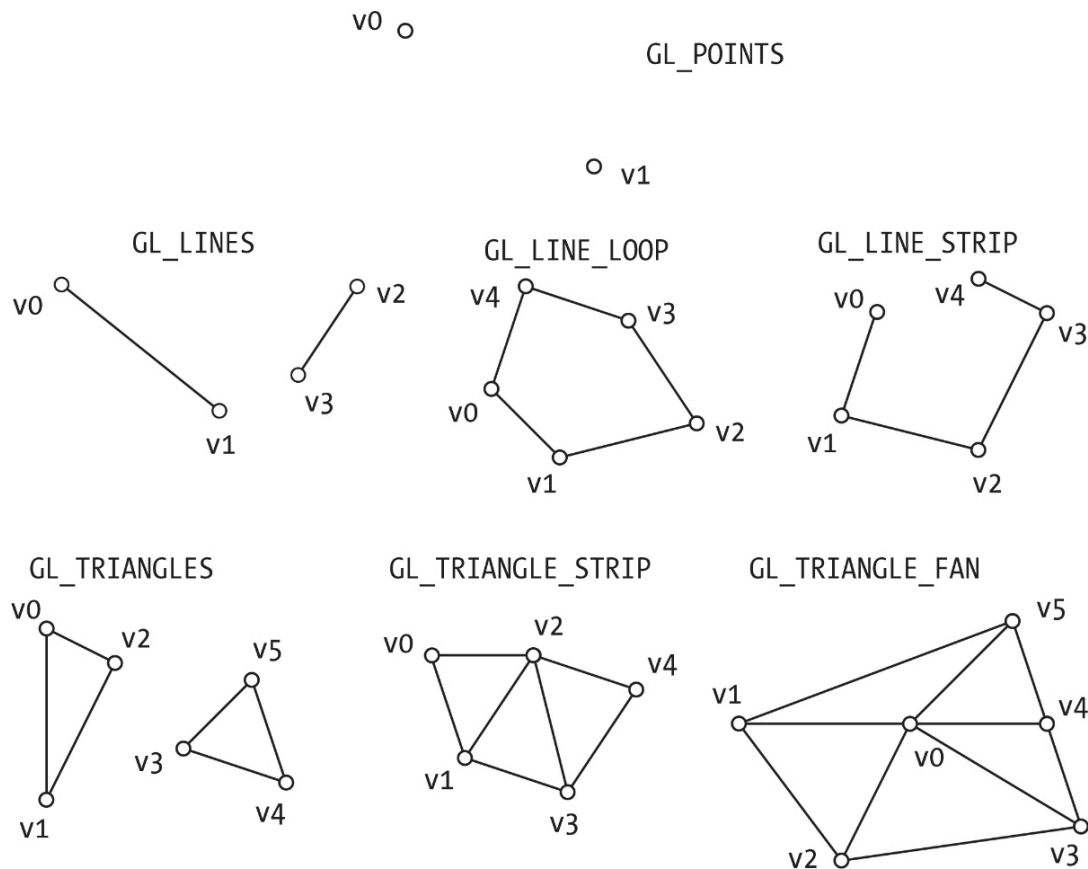


Figure 9-3: OpenGL primitives

To draw a sphere in OpenGL, first define the geometry of the sphere mathematically and compute its 3D vertices. Then assemble the vertices into basic geometric primitives; for example, you could group each set of three vertices into a triangle. You then render the vertices using OpenGL.

3D Transformations

You can't learn computer graphics without learning about 3D transformations. Conceptually, these are quite simple to understand. You have an object—what can you do to it? You can move it, stretch (or squash) it, or rotate it. You can do other things to it too, but these three tasks—translation, scale, and rotation—are the operations or transformations most commonly performed on an object. In addition to these commonly used transformations, you'll use a perspective projection to map the 3D objects onto the 2D plane of the screen. These transforma-

tions are all applied on the coordinates of the object you are trying to transform.

While you're probably familiar with 3D coordinates in the form (x, y, z) , in 3D computer graphics you use coordinates in the form (x, y, z, w) , called *homogeneous coordinates*. (These coordinates come from a branch of mathematics called *projective geometry*, which is beyond the scope of this book.) Homogeneous coordinates allow you to express common 3D transformations such as translation, scale, and rotation as 4×4 matrices. But for the purposes of these OpenGL projects, all you need to know is that the homogeneous coordinate (x, y, z, w) is equivalent to the 3D coordinate $(x/w, y/w, z/w, 1.0)$. A 3D point $(1.0, 2.0, 3.0)$ can be expressed in homogeneous coordinates as $(1.0, 2.0, 3.0, 1.0)$.

Here's an example of a 3D transformation using a 4×4 matrix.

See how the matrix multiplication translates a point $(x, y, z, 1.0)$ to $(x + t_x, y + t_y, z + t_z, 1.0)$:

$$\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{bmatrix}$$

Since this operation translates a point in space, the 4×4 matrix involved is called a *translation matrix*.

Let's now look at another useful matrix for 3D transformation—a rotation matrix. The following matrix rotates a point $(x, y, z, 1.0)$ counter-clockwise around the x-axis by θ radians:

$$R_{\theta,x} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

But here's something to keep in mind: if you're going to apply this rotation inside shader code, the matrix will be stored in *column-major format*, which means you should declare it as follows:

```
// rotational transform

mat4 rot = mat4(

    vec4(1.0, 0.0,    0.0,    0.0),

    vec4(0.0, cos(uTheta), sin(uTheta), 0.0),

    vec4(0.0, -sin(uTheta), cos(uTheta), 0.0),

    vec4(0.0, 0.0,    0.0,    1.0)

);
```

Notice that, in the code, the matrix is flipped along its diagonal compared to the definition of $R_{\theta,x}$.

Two terms that you will encounter often in OpenGL are *modelview* and *projection* transformations. With the advent of customizable shaders in modern OpenGL, modelviews and projections are just generic transformations. Historically, in old-school versions of OpenGL, the modelview transformations were applied to your 3D model to position it in space, and the projection transformations were

used to map the 3D coordinates onto a 2D surface for display, as you'll see in a moment. Modelview transformations are user-defined transformations that let you position your 3D objects, and projection transformations are projective transformations that map 3D onto 2D.

The two most commonly used 3D graphics projective transformations are *orthographic* and *perspective*, but here you'll use only perspective projections, which are defined by a *field of view* (the extent to which the eye can see), a *near plane* (the plane closest to the eye), a *far plane* (the plane farthest from the eye), and an *aspect ratio* (the ratio of the width to the height of the near plane). Together, these parameters constitute a camera model for a projection that determines how the 3D figure will be mapped onto a 2D screen, as shown in [Figure 9-4](#). The truncated pyramid shown in the figure is the *view frustum*. The *eye* is the 3D location where you place the camera. (For orthographic projection, the eye will be at infinity, and the pyramid will become a rectangular cuboid.)

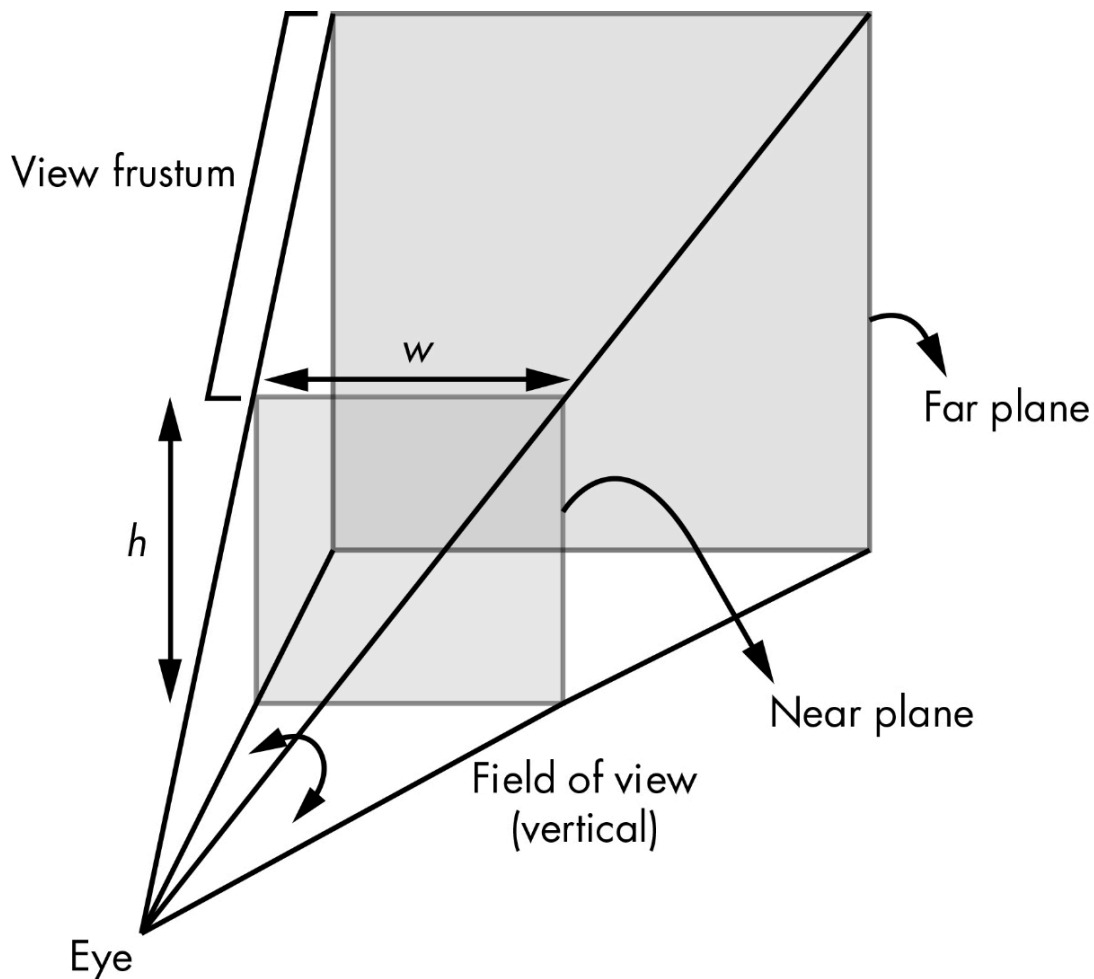


Figure 9-4: A perspective projection camera model

Once the perspective projection is complete and before rasterization, the graphics primitives are clipped (or cut out) against the near and far planes shown in [Figure 9-4](#). The near and far planes are chosen such that the 3D objects you want to appear onscreen lie inside the view frustum; otherwise, they will be clipped away.

Shaders

You've seen how shaders fit into the modern OpenGL programmable graphics pipeline. Now let's look at a simple pair of vertex and fragment shaders to get a sense of how GLSL works.

A Vertex Shader

Here is a simple vertex shader that computes the position and color of a vertex:

```
❶ # version 410 core

❷ in vec3 aVert;

❸ uniform mat4 uMVMatrix;

❹ uniform mat4 uPMatrix;

❺ out vec4 vCol;

void main() {

    // apply transformations

    ❻ gl_Position = uPMatrix * uMVMatrix * vec4(aVert, 1.0);

    // set color

    ❼ vCol = vec4(1.0, 0.0, 0.0, 1.0);

}
```

You first set the version of GLSL used in the shader to version 4.1 ❶. Then you define an input named `aVert` of type `vec3` (a 3D vector) for the vertex shader using the keyword `in` ❷. You next define two variables of type `mat4` (4×4 matrices), which correspond to the modelview ❸ and projection ❹ matrices. The `uniform` prefix to these variables indicates that they do not change during execution of the vertex shader for a given rendering call on a set of vertices. You use the `out` prefix to define the output of the vertex shader, which is a color variable of type `vec4` (a 4D vector to store red, green, blue, and alpha channels) ❺.

Now you come to the `main()` function, where the vertex shader program starts. The value of `gl_Position` is computed by transforming the input `aVert` using the uniform matrices passed in ❹. The GLSL variable `gl_Position` is used to store the transformed vertices. You set the output color from the vertex shader to red with no transparency by using the value `(1, 0, 0, 1)` ❺. You'll use this as input in the next shader in the pipeline.

A Fragment Shader

Now let's look at a simple fragment shader that computes the fragment color based on the vertex color passed in:

```
❶ # version 410 core
```

```
❷ in vec4 vCol;
```

```
❸ out vec4 fragColor;
```

```
void main() {
```

```
    // use vertex color
```

```
    ❹ fragColor = vCol;
```

```
}
```

After setting the version of GLSL used in the shader ❶, you set `vCol` as the input to the fragment shader ❷. This variable, `vCol`, was set as output from the vertex shader. (Remember, the vertex shader executes for every vertex in the 3D scene, whereas the fragment shader executes for every fragment on the screen.) You also set the fragment shader's output color variable `fragColor` ❸.

During rasterization (which occurs between the vertex and fragment shaders), OpenGL converts the transformed vertices to fragments, and

the color of the fragments lying between the vertices is calculated by interpolating the color values at the vertices; `vCol` in the previous code is this interpolated color. You set the fragment shader's output to be the same as the interpolated color going into the fragment shader ④. By default, and in most cases, the intended output of the fragment shader is the screen, and the color you set ends up there (unless it's affected by operations, such as depth testing, that occur in the final stage of the graphics pipeline).

For the GPU to execute the shader code, it needs to be compiled and linked to instructions that the hardware understands. OpenGL provides ways to do this and reports detailed compiler and linker errors that will help you develop the shader code. The compilation process also generates a table of locations or indices for the variables declared in your shaders so you can connect them to variables in your Python code.

Vertex Buffers

Vertex buffers are an important mechanism used by OpenGL shaders. Modern graphics hardware and OpenGL are designed to work with large amounts of 3D geometry. Consequently, several mechanisms are built into OpenGL to help transfer data from the program to the GPU. A typical setup to draw 3D geometry in a program will do the following:

1. Define arrays of coordinates, colors, and other attributes for each vertex of the 3D geometry.
2. Create a vertex array object (VAO) and bind to it.
3. Create vertex buffer objects (VBOs) for each attribute, defined on a per-vertex basis.
4. Bind to the VBO and set the buffer data using the predefined arrays.
5. Specify the data and location of vertex attributes to be used in the shader.

6. Enable the vertex attributes.
7. Render the data.

After you define the 3D geometry in terms of vertices, you create and bind to a vertex array object. VAOs are a convenient way to group geometry as multiple arrays of coordinates, colors, and so on. Then, for each attribute of each vertex, you create a vertex buffer object and set your 3D data into it. The VBO stores the vertex data in the GPU memory. Now, all that's left is to connect the buffer data so you can access it from your shaders. You do this through calls that use the location of the variables employed in the shader.

Texture Mapping

Now let's look at texture mapping, an important computer graphics technique that you'll use in this chapter. *Texture mapping* is a way to give a scene a realistic feel with the help of a 2D picture of a 3D object (like the backdrop in a play). A texture is usually read from an image file and is stretched to drape over a geometric region by mapping the 2D coordinates (in the range [0, 1]) onto the 3D coordinates of the polygons. For example, **Figure 9-5** shows an image draped onto one face of a cube. (I used `GL_TRIANGLE_STRIP` primitives to draw the cube faces, and the ordering of the vertices is indicated by the lines on the face.)

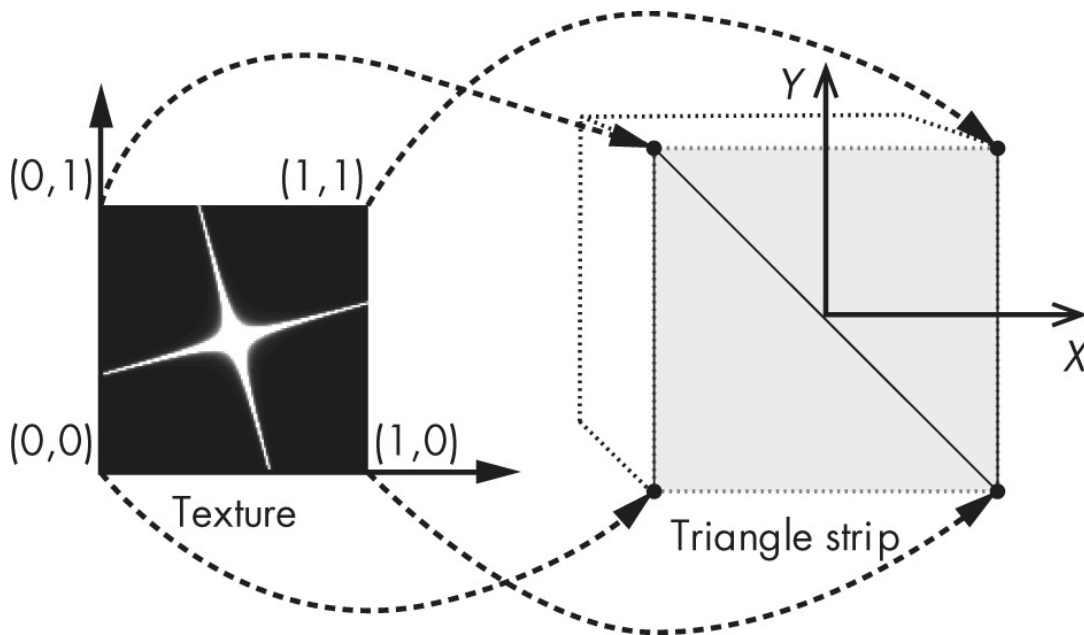


Figure 9-5: *Texture mapping*

In **Figure 9-5**, the (0, 0) corner of the texture is mapped to the bottom-left vertex of the cube face. Similarly, you can see how the other corners of the texture are mapped, with the net effect that the texture is “pasted” onto this cube face. The geometry of the cube face itself is defined as a triangle strip, and the vertices zigzag from the bottom to the top left and from the bottom to the top right. Textures are extremely powerful and versatile computer graphics tools, as you’ll see in **Chapter 11**.

The OpenGL Context

Now let’s talk about how to get OpenGL to draw stuff on the screen. The entity that stores all the OpenGL state information is called the *OpenGL context*. Contexts have a viewable, window-like area where the OpenGL drawings go, and you can have multiple contexts per process or run of an application, but only one context per thread can be current at a time. (Fortunately, the window toolkit will take care of most of the context handling.)

For your OpenGL output to appear in a window onscreen, you need the help of the operating system. For these projects, you'll use GLFW, a lightweight cross-platform C library that lets you create and manage OpenGL contexts, display the 3D graphics in a window, and handle user input such as mouse clicks and keypresses. ([Appendix A](#) covers the installation details for this library.)

Because you're writing code in Python and not C, you'll also use a Python binding to GLFW (*glfw.py*, available in the *common* directory in the book's code repository), which lets you access all the GLFW features using Python.

Requirements

You'll use `PyOpenGL`, a popular Python binding for OpenGL, for rendering, and you'll use `numpy` arrays to represent 3D coordinates and transformation matrices.

The Code

In this project, you'll build a simple Python application for displaying a rotating, textured polygon using OpenGL. To see the complete project code, skip ahead to [“The Complete Code”](#) on [page 172](#). The complete code for our simple OpenGL application resides in two files. The main project code discussed in this chapter is in *simpleglfw.py*, which can be found at <https://github.com/mkvenkit/pp2e/tree/main/simplegl>. The helper functions are in *glutils.py*, which can be found in the GitHub repository's *common* directory.

The RenderWindow Class

The `RenderWindow` class manages the creation of the window that displays the OpenGL graphics. It initializes GLFW, sets up OpenGL, manages rendering, and sets up callbacks to receive keyboard input.

Creating an OpenGL Window

The first order of business of the `RenderWindow` class is to set up GLFW so you have an OpenGL window to render into. The class's initialization code addresses this task:

```
class RenderWindow:
```

```
    """GLFW rendering window class"""
```

```
    def __init__(self):
```

```
        # save current working directory
```

```
        cwd = os.getcwd()
```

```
        # initialize glfw
```

```
        ❶ glfw.glfwInit()
```

```
        # restore cwd
```

```
        os.chdir(cwd)
```

```
        # version hints
```

```
        ❷ glfw.glfwWindowHint(glfw.GLFW_CONTEXT_VERSION_MAJOR,  
4)
```

```
        glfw.glfwWindowHint(glfw.GLFW_CONTEXT_VERSION_MINOR, 1)
```

```
        glfw.glfwWindowHint(glfw.GLFW_OPENGL_FORWARD_COMPAT,  
GL_TRUE)
```

```
        glfw.glfwWindowHint(glfw.GLFW_OPENGL_PROFILE,
```

```
            glfw.GLFW_OPENGL_CORE_PROFILE)
```



```
# make a window

self.width, self.height = 800, 600

self.aspect = self.width/float(self.height)

❸ self.win = glfw.glfwCreateWindow(self.width, self.height,

                                   b'simpleglfw')

# make the context current

❹ glfw.glfwMakeContextCurrent(self.win)
```

You initialize the GLFW library ❶, and then, starting at ❷, you set the OpenGL version to the OpenGL 4.1 core profile. You next create an OpenGL-capable window with the dimensions 800×600 ❸. Finally, you make the context current ❹, and you're ready to make OpenGL calls.

Next, still within the `__init__()` definition, you make some initialization calls:

```
# initialize GL

❶ glViewport(0, 0, self.width, self.height)

❷ glEnable(GL_DEPTH_TEST)

❸ glClearColor(0.5, 0.5, 0.5, 1.0)
```

Here you set the viewport or screen dimensions (width and height) where OpenGL will render your 3D scene ❶. Then you turn on depth testing with `GL_DEPTH_TEST` ❷ and set the color the background should become when `glClearColor()` is issued during rendering ❸. You choose 50 percent gray with an `alpha` setting of 1.0. (Alpha is a measure of the transparency of a fragment—1.0 means fully opaque.)

Setting Callbacks

You finish the `__init__()` definition by registering event callbacks for user interface events within the GLFW window so you can respond to keypresses:

```
# set window callbacks
```

```
glfw.glfwSetKeyCallback(self.win, self.onKeyboard)
```

This code sets callbacks for keypresses. Every time one of these events happens, the function registered as a callback, `onKeyboard()`, is executed. Let's look at the definition of that keyboard callback function now:

```
def onKeyboard(self, win, key, scancode, action, mods):
```

```
    # print 'keyboard: ', win, key, scancode, action, mods
```

```
    ❶ if action == glfw.GLFW_PRESS:
```

```
        # ESC to quit
```

```
        if key == glfw.GLFW_KEY_ESCAPE:
```

```
            ❷ self.exitNow = True
```

```
        else:
```

```
            # toggle cut
```

```
            ❸ self.scene.showCircle = not self.scene.showCircle
```

The `onKeyboard()` callback is called every time a keyboard event happens. The arguments to the function arrive filled with useful information such as what type of event occurred (key-up versus key-down, for example) and which key was pressed. The code `glfw.GLFW_PRESS`

says to look only for key-down, or `PRESS`, events ❶. You set an exit flag if the ESC key is pressed ❷. If any other key is pressed, you toggle a `showCircle` Boolean ❸. This variable will be used in the fragment shader to keep or discard fragments outside the circle area.

Defining the Main Loop

The `RenderWindow` class also defines the main loop of the program through its `run()` method. (GLFW doesn't provide a default program loop.) The `run()` method updates the OpenGL window at a preset time interval. After calling the render methods to draw the scene, it also polls the system for any pending window or keyboard events. Let's look at the method definition:

```
def run(self):
```

```
    # initializer timer
```

```
    ❶ glfw.glfwSetTime(0)
```

```
    t = 0.0
```

```
    ❷ while not glfw.glfwWindowShouldClose(self.win) and not  
self.exitNow:
```

```
        # update every x seconds
```

```
        ❸ currT = glfw.glfwGetTime()
```

```
        if currT - t > 0.1:
```

```
            # update time
```

```
            t = currT
```

```
            # clear
```

④ `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`

`# set viewport`

⑤ `self.width, self.height = glfw.glfwGetFramebufferSize(self.win)`

⑥ `self.aspect = self.width/float(self.height)`

⑦ `glViewport(0, 0, self.width, self.height)`

In the main loop, `glfw.glfwSetTime()` resets the GLFW timer to 0 ①. You'll use this timer to redraw the graphics at regular intervals. You initiate a `while` loop ② that exits only if the window is closed or `exitNow` is set to `True`. When the loop exits, `glfw.glfwTerminate()` is called to shut down GLFW cleanly.

Inside the loop, `glfw.glfwGetTime()` gets the current timer value ③, which you use to calculate the elapsed time since the last drawing. By setting a desired interval here (in this case, to 0.1 second or 100 milliseconds), you can adjust the rendering frame rate. Next, `glClear()` clears the depth and color buffers and replaces them with the set background color to get ready for the next frame ④.

You query and set the window width and height using the `glfwGetFramebufferSize()` function ⑤. You do this in case the user has changed the window's size. Note that in some systems (such as a MacBook with Retina display) the window size and frame buffer size can be different, so to be safe, always query the latter. You next compute the aspect ratio of the window ⑥, which you'll use later to set the projection matrix. Then you clear the viewport using the new frame buffer dimensions you retrieved ⑦.

Now let's look at the remaining part of the `run()` method:

`# build projection matrix`

```
❶ pMatrix = glutils.perspective(45.0, self.aspect, 0.1, 100.0)

❷ mvMatrix = glutils.lookAt([0.0, 0.0, -2.0], [0.0, 0.0, 0.0],

                             [0.0, 1.0, 0.0])

# render

❸ self.scene.render(pMatrix, mvMatrix)

# step

❹ self.scene.step()

❺ glfw.glfwSwapBuffers(self.win)

# poll for and process events

❻ glfw.glfwPollEvents()

# end

glfw.glfwTerminate()
```

Still within the `while` loop, you compute the projection matrix using the `perspective()` method defined in `glutils.py` ❶. The projection matrix is the transformation that maps a 3D scene to a 2D screen. Here you ask for a 45-degree field of view and a near/far plane distance of 0.1/100.0. Then you set the modelview matrix using the `lookAt()` method ❷, also defined in `glutils.py`. The default OpenGL view puts your eye at the origin looking along the negative z-direction. The modelview matrix created by the `lookAt()` method transforms the vertices such that the view matches up with the eye position and orientation specified by the call. You set the eye position to (0, 0, -2), looking at the origin (0, 0, 0) with an “up” vector of (0, 1, 0). Next, you call the `render()` method on the `scene` object ❸, passing in these matrices,

and you call `scene.step()` so it can update the variables necessary for the time step ❷. (The `Scene` class, which we'll look at next, encapsulates the setup and rendering of the polygon.) The `glfwSwapBuffers()` call ❸ swaps the back and front buffers, thus displaying your updated 3D graphic, and the `glfwPollEvents()` call ❹ checks for any UI events and returns control to the `while` loop.

The Scene Class

Now let's look at the `Scene` class, which is responsible for initializing and drawing the 3D geometry. Here's the start of the class declaration:

```
class Scene:
```

```
    """ OpenGL 3D scene class """
```

```
    # initialization
```

```
    def __init__(self):
```

```
        # create shader
```

```
        ❶ self.program = glutils.loadShaders(strVS, strFS)
```

```
        ❷ glUseProgram(self.program)
```

In the `Scene` class constructor, you first compile and load the shaders. For this, you use the utility method `loadShaders()` ❶ defined in `glutils.py`, which provides a convenient wrapper around the series of OpenGL calls required to load the shader code from strings, compile it, and link it into an OpenGL program object. Because OpenGL is a state machine, you need to set the code to use a particular “program object” (because a project could have multiple programs) using the `glUseProgram()` call ❷.

The `__init__()` method continues by connecting the variables in the Python code with those in the shaders:

```
self.pMatrixUniform = glGetUniformLocation(self.program,
b'uPMatrix')

self.mvMatrixUniform = glGetUniformLocation(self.program,
b'uMVMMatrix')

# texture

self.tex2D = glGetUniformLocation(self.program, b'tex2D')
```

This code uses the `glGetUniformLocation()` method to retrieve the locations of the variables `uPMatrix`, `uMVMMatrix`, and `tex2D` defined inside the vertex and fragment shaders. These locations can then be used to set the values for the shader variables.

Defining the 3D Geometry

The next part of the `Scene` class's `__init__()` method defines the 3D geometry for the scene. You first define the geometry for the polygon, which will take the form of a square:

```
# define triangle strip vertices

❶ vertexData = numpy.array(

    [-0.5, -0.5, 0.0,

     0.5, -0.5, 0.0,

     -0.5, 0.5, 0.0,

     0.5, 0.5, 0.0], numpy.float32)

# set up vertex array object (VAO)
```

```
❷ self.vao = glGenVertexArrays(1)

glBindVertexArray(self.vao)

# vertices

❸ self.vertexBuffer = glGenBuffers(1)

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

# set buffer data

❹ glBufferData(GL_ARRAY_BUFFER, 4*len(vertexData), vertexData,

               GL_STATIC_DRAW)

# enable vertex array

❺ glEnableVertexAttribArray(0)

# set buffer data pointer

❻ glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, None)

# unbind VAO

❼ glBindVertexArray(0)
```

First you define the array of vertices of the triangle strip used to draw the square ❶. Think of a square of a side length of 1.0 centered at the origin. The bottom-left vertex of this square has the coordinates $(-0.5, -0.5, 0.0)$; the next vertex (the bottom-right one) has the coordinates $(0.5, -0.5, 0.0)$; and so on. The order of the four coordinates is that of a `GL_TRIANGLE_STRIP`. Essentially, you're creating the square by defining two right triangles with a shared hypotenuse.

Next, you create a VAO ❷. Once you bind to this VAO, all upcoming calls will be bound to it. You then create a VBO to manage the rendering of the vertex data ❸. Once the buffer is bound, you set the buffer data from the vertices you've defined ❹.

Now you need to enable the shaders to access this data. For that, you call `glEnableVertexAttribArray()` ❺. You use an index of 0 because that is the location you have set in the vertex shader for the vertex data variable. Calling `glVertexAttribPointer()` sets the location and data format of the vertex attribute array ❻. The index of the attribute is 0, the number of components is 3 (you use 3D vertices), and the data type of the vertex is `GL_FLOAT`. You then unbind the VAO ❼ so other related calls don't interfere with it. In OpenGL, it's a best practice to reset states when you're done. OpenGL is a state machine, so if you leave things in a mess, they will remain that way.

The following code loads an image of a star as an OpenGL texture:

```
# texture

self.texId = glutils.loadTexture('star.png')
```

The texture ID returned will be used later in rendering.

Rotating the Square

Next you need to update variables in the `Scene` object to make the square rotate on the screen. Use the class's `step()` method:

```
# step

def step(self):

    # increment angle
```

```
❶ self.t = (self.t + 1) % 360
```

At ❶, you increment the angle variable `t` and use the modulus operator (`%`) to keep this value within `[0, 360]`. This variable will be used to update the rotation angle in the vertex shader.

Rendering the Scene

Now let's look at the `Scene` object's main rendering code:

```
def render(self, pMatrix, mvMatrix):
```

```
    # use shader
```

```
    ❶ glUseProgram(self.program)
```

```
    # set projection matrix
```

```
    ❷ glUniformMatrix4fv(self.pMatrixUniform, 1, GL_FALSE, pMatrix)
```

```
    # set modelview matrix
```

```
    glUniformMatrix4fv(self.mvMatrixUniform, 1, GL_FALSE, mvMatrix)
```

```
    # set shader angle in radians
```

```
    ❸ glUniform1f(glGetUniformLocation(self.program, 'uTheta'),
```

```
        math.radians(self.t))
```

```
    # show circle?
```

```
    ❹ glUniform1i(glGetUniformLocation(self.program, b'showCircle'),
```

```
        self.showCircle)
```

```
    # enable texture
```

```
❶ glActiveTexture(GL_TEXTURE0)

❷ glBindTexture(GL_TEXTURE_2D, self.texId)

❸ glUniform1i(self.tex2D, 0)

# bind VAO

❹ glBindVertexArray(self.vao)

# draw

❺ glDrawArrays(GL_TRIANGLE_STRIP, 0, 4)

# unbind VAO

❻ glBindVertexArray(0)
```

First you set up the rendering to use the shader program ❶. Starting at ❷, you set the computed projection and modelview matrices in the shader using the `glUniformMatrix4fv()` method. Then you use the `glUniform1f()` method to set `uTheta` in the shader program ❸. You use `glGetUniformLocation()` as before to get the location of the `uTheta` angle variable from the shader, and you use the Python `math.radians()` method to convert the angle from degrees to radians. Next, you use `glUniform1i()` to set the current value of the `showCircle` variable in the fragment shader ❹. OpenGL has a concept of multiple texture units, and `glActiveTexture()` ❺ activates texture unit 0 (the default). You bind the texture ID you generated earlier from the *star.png* image to activate it for rendering ❻. The `sampler2D` variable in the fragment shader is set to texture unit 0 ❼.

You continue by binding to the VAO you created previously ❽. Now you see the benefit of using VAOs: you don't need to repeat a whole bunch of vertex buffer-related calls before the actual drawing. You then call `glDrawArrays()` to render the bound vertex buffers ❾. The

primitive type is a triangle strip, and there are four vertices to be rendered. Finally, you unbind the VAO at ⑩, which is always a good coding practice.

Defining the GLSL Shaders

Now let's look at the most exciting part of the project—the GLSL shaders. First, here's the vertex shader, which computes the position and texture coordinates of the vertices:

```
# version 410 core
```

```
① layout(location = 0) in vec3 aVert;
```

```
② uniform mat4 uMVMMatrix;
```

```
uniform mat4 uPMatrix;
```

```
uniform float uTheta;
```

```
③ out vec2 vTexCoord;
```

```
void main() {
```

```
    // rotational transform
```

```
④ mat4 rot = mat4(
```

```
    vec4(1.0, 0.0, 0.0, 0.0),
```

```
    vec4(0.0, cos(uTheta), -sin(uTheta), 0.0),
```

```
    vec4(0.0, sin(uTheta), cos(uTheta), 0.0),
```

```
    vec4(0.0, 0.0, 0.0, 1.0)
```

```
);
```

```

// transform vertex

❶ gl_Position = uPMatrix * uMVMMatrix * rot * vec4(aVert, 1.0);

// set texture coordinate

❷ vTexCoord = aVert.xy + vec2(0.5, 0.5);

}

```

You use the `layout` keyword ❶ to set explicitly the location of the vertex attribute `aVert`—to 0, in this case. This attribute lets the vertex shader access the vertices that you defined for the polygon. Starting at ❷, you declare three `uniform` variables for the projection and modelview matrices and the rotation angle. These will be set from the Python code. You also set a 2D vector `vTexCoord` as an output from this shader ❸. This will be available as an input to the fragment shader.

In the `main()` method in the shader, you set up a rotation matrix ❹, which rotates around the x-axis by a given angle, `uTheta`. You compute `gl_Position` ❺ using a concatenation of the projection, modelview, and rotation matrices. This gives you the position of the output vertex from the shader. You then set up a 2D vector as a texture coordinate ❻. You may recall that you defined the triangle strip for a square centered at the origin with side 1.0. Because texture coordinates are in the range [0, 1], you can generate these from the vertex coordinates by adding (0.5, 0.5) to the x- and y-values. This also demonstrates the power and immense flexibility of shaders for your computations. Texture coordinates and other variables aren't sacrosanct; you can set them to just about anything.

Now let's look at the fragment shader, which computes the output pixels of our OpenGL program:

```
# version 410 core
```

❶ in vec2 vTexCoord;

❷ uniform sampler2D tex2D;

❸ uniform bool showCircle;

❹ out vec4 fragColor;

void main() {

if (showCircle) {

// discard fragment outside circle

❺ if (distance(vTexCoord, vec2(0.5, 0.5)) > 0.5) {

discard;

}

else {

❻ fragColor = texture(tex2D, vTexCoord);

}

}

else {

❼ fragColor = texture(tex2D, vTexCoord);

}

}

You start by defining inputs to the fragment shader—in this case the texture coordinate you set as output in the vertex shader ❶. Recall that the fragment shader operates on a per-pixel basis, so the values set for these variables are those for the current pixel, interpolated across the polygon. You declare a `sampler2D` variable ❷, which is linked to a particular texture unit and is used to look up the texture value, and a Boolean uniform flag `showCircle` ❸, which is set from the Python code. You also declare `fragColor` as the output from the fragment shader ❹. By default, this goes to the screen (after final frame buffer operations such as depth testing and blending).

Within the `main()` method, if the `showCircle` flag is not set ❺, you use the GLSL `texture()` method to look up the texture color value using the texture coordinate and the sampler. In effect, you're just texturing the triangle strip using the star image. If, however, the `showCircle` flag is `true` ❻, you use the GLSL built-in method `distance()` to check how far the current pixel is from the center of the polygon. It uses the (interpolated) texture coordinates for this purpose, which are passed in by the vertex shader. If the distance is greater than a certain threshold (0.5 in this case), you call the GLSL `discard()` method, which drops the current pixel. If the distance is less than the threshold, you set the appropriate color from the texture ❼. Basically, what this does is ignore pixels that are outside a circle with a radius of 0.5 centered at the midpoint of the square, thus cutting the polygon into a circle when `showCircle` is set.

Utility Functions

I've referred to several utility functions defined for you in *glutils.py* to make working with OpenGL easier. Let's look at an example of one of those functions now. The `loadTexture()` function loads an image into an OpenGL texture:

```
def loadTexture(filename):
```

```
"""load OpenGL 2D texture from given image file"""

❶ img = Image.open(filename)

❷ imgData = numpy.array(list(img.getdata()), np.int8)

❸ texture = glGenTextures(1)

❹ glBindTexture(GL_TEXTURE_2D, texture)

❺ glPixelStorei(GL_UNPACK_ALIGNMENT, 1)

❻ glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE)

glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE)

❼ glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR)

glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR)

❽ glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, img.size[0],
img.size[1],

0, GL_RGBA, GL_UNSIGNED_BYTE, imgData)

return texture
```

The `loadTexture()` function uses the Python Imaging Library (PIL) `Image` module to read the image file ❶. Then it gets the data out of the `Image` object onto an 8-bit `numpy` array ❷ and creates an OpenGL `texture` object ❸, which is a prerequisite to doing anything with textures in OpenGL. You next perform the now familiar binding to the

`texture` object ④ so all further texture-related settings apply to this object. You set the unpacking alignment of data to 1 ⑤, which means the image data will be considered to be 1-byte or 8-bit data by the hardware. Starting at ⑥, you tell OpenGL what to do with the texture at the edges. In this case, you direct it to just clamp the texture color to the edge of the geometry. (In specifying texture coordinates, the convention is to use the letters `S` and `T` for the axes instead of `x` and `y`.) At ⑦ and the following line, you specify the kind of interpolation to be used when the texture is stretched or compressed to map onto a polygon. In this case, *linear filtering* is specified. Finally, you set the image data in the bound texture ⑧. At this point, the image data is transferred to graphics memory, and the texture is ready for use.

Running the OpenGL Application

Here is a sample run of the project:

```
$ python simpleglfw.py
```

You saw the output in [Figure 9-1](#). Be sure to try some keypresses to toggle the circle on and off.

Summary

Congratulations on completing your first program using Python and OpenGL! Through this project, you've learned about creating 3D transformations, using the OpenGL 3D graphics pipeline, and using GLSL vertex and fragment shaders to create interesting 3D graphics. You've begun your journey into the fascinating world of 3D graphics programming.

Experiments!

Here are some ideas for modifying this project:

1. The vertex shader in this project rotates the square around the x-axis (1, 0, 0). Can you make it rotate around the y-axis (0, 0, 1)? You can do this in one of two ways: first, by modifying the rotation matrix in the shader, or second, by computing this matrix in the Python code and passing it as a *uniform* into the shader. Try both!
2. In the project, the texture coordinates are generated inside the vertex shader and passed to the fragment shader. This is a trick, and it works only because of the convenient values chosen for the vertices of the triangle strip. Pass the texture coordinates as a separate attribute into the vertex shader, similar to how the vertices are passed in. Now, can you make the star texture *tile* across the triangle strip? Instead of displaying a single star, you want to produce a 4×4 grid of stars on the square. (Hint: use texture coordinates greater than 1.0 and set `GL_TEXTURE_WRAP_S/T` parameters in `glTexParameterf()` to `GL_REPEAT`.)
3. By changing just your fragment shader, can you make your square look like **Figure 9-6**? (Hint: use the GLSL `sin()` function.)



Figure 9-6: Using the fragment shader to block out concentric circles

The Complete Code

Here's the complete *simpleglfw.py* code:

```
"""
```

```
simpleglfw.py
```

A simple Python OpenGL program that uses PyOpenGL + GLFW to get an

OpenGL 4.1 context.

Author: Mahesh Venkitachalam

```
"""
```

```
import OpenGL
```

```
from OpenGL.GL import *
```

```
import numpy, math, sys, os
```

```
import glutils
```

```
import glfw
```

```
strVS = """
```

```
# version 410 core
```

```
layout(location = 0) in vec3 aVert;
```

```
uniform mat4 uMVMatrix;
```

```
uniform mat4 uPMatrix;
```

```
uniform float uTheta;

out vec2 vTexCoord;

void main() {

    // rotational transform

    mat4 rot = mat4(

        vec4(1.0, 0.0,      0.0,      0.0),

        vec4(0.0, cos(uTheta), sin(uTheta), 0.0),

        vec4(0.0, -sin(uTheta), cos(uTheta), 0.0),

        vec4(0.0, 0.0,      0.0,      1.0)

    );

    // transform vertex

    gl_Position = uPMatrix * uMVMatrix * rot * vec4(aVert, 1.0);

    // set texture coord

    vTexCoord = aVert.xy + vec2(0.5, 0.5);

}

"""

strFS = """

# version 410 core

in vec2 vTexCoord;
```

```
uniform sampler2D tex2D;

uniform bool showCircle;

out vec4 fragColor;

void main() {

    if (showCircle) {

        // discard fragment outside circle

        if (distance(vTexCoord, vec2(0.5, 0.5)) > 0.5) {

            discard;

        }

        else {

            fragColor = texture(tex2D, vTexCoord);

        }

    }

    else {

        fragColor = texture(tex2D, vTexCoord);

    }

}

}

"""

class Scene:
```

```
""" OpenGL 3D scene class """

# initialization

def __init__(self):

    # create shader

    self.program = glutils.loadShaders(strVS, strFS)

    glUseProgram(self.program)

    self.pMatrixUniform = glGetUniformLocation(self.program,

                                                b'uPMatrix')

    self.mvMatrixUniform = glGetUniformLocation(self.program,

                                                b'uMVMatrix')

    # texture

    self.tex2D = glGetUniformLocation(self.program, b'tex2D')

    # define triangle strip vertices

    vertexData = numpy.array(

        [-0.5, -0.5, 0.0,

         0.5, -0.5, 0.0,

         -0.5, 0.5, 0.0,

         0.5, 0.5, 0.0], numpy.float32)

    # set up vertex array object (VAO)
```

```
self.vao = glGenVertexArrays(1)

glBindVertexArray(self.vao)

# vertices

self.vertexBuffer = glGenBuffers(1)

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

# set buffer data

glBufferData(GL_ARRAY_BUFFER, 4*len(vertexData), vertexData,

             GL_STATIC_DRAW)

# enable vertex array

glEnableVertexAttribArray(0)

# set buffer data pointer

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, None)

# unbind VAO

glBindVertexArray(0)

# time

self.t = 0

# texture

self.texId = glutils.loadTexture('star.png')

# show circle?
```

```
self.showCircle = False

# step

def step(self):

    # increment angle

    self.t = (self.t + 1) % 360

# render

def render(self, pMatrix, mvMatrix):

    # use shader

    glUseProgram(self.program)

    # set proj matrix

    glUniformMatrix4fv(self.pMatrixUniform, 1, GL_FALSE, pMatrix)

    # set modelview matrix

    glUniformMatrix4fv(self.mvMatrixUniform, 1, GL_FALSE,
mvMatrix)

    # set shader angle in radians

    glUniform1f(glGetUniformLocation(self.program, 'uTheta'),

        math.radians(self.t))

    # show circle?

    glUniform1i(glGetUniformLocation(self.program, b'showCircle'),
```



```
        self.showCircle)

    # enable texture

    glActiveTexture(GL_TEXTURE0)

    glBindTexture(GL_TEXTURE_2D, self.texId)

    glUniform1i(self.tex2D, 0)

    # bind VAO

    glBindVertexArray(self.vao)

    # draw

    glDrawArrays(GL_TRIANGLE_STRIP, 0, 4)

    # unbind VAO

    glBindVertexArray(0)

class RenderWindow:

    """GLFW rendering window class"""

    def __init__(self):

        # save current working directory

        cwd = os.getcwd()

        # initialize glfw - this changes cwd

        glfw.glfwInit()

        # restore cwd
```

```
os.chdir(cwd)

# version hints

glfw.glfwWindowHint(glfw.GFW_CONTEXT_VERSION_MAJOR, 4)

glfw.glfwWindowHint(glfw.GFW_CONTEXT_VERSION_MINOR, 1)

glfw.glfwWindowHint(glfw.GFW_OPENGL_FORWARD_COMPAT,
GL_TRUE)

glfw.glfwWindowHint(glfw.GFW_OPENGL_PROFILE,

                    glfw.GFW_OPENGL_CORE_PROFILE)

# make a window

self.width, self.height = 800, 600

self.aspect = self.width/float(self.height)

self.win = glfw.glfwCreateWindow(self.width, self.height,

                                b'simpleglfw')

# make context current

glfw.glfwMakeContextCurrent(self.win)

# initialize GL

glViewport(0, 0, self.width, self.height)

glEnable(GL_DEPTH_TEST)

glClearColor(0.5, 0.5, 0.5, 1.0)
```

```
# set window callbacks

glfw.glfwSetKeyCallback(self.win, self.onKeyboard)

# create 3D

self.scene = Scene()

# exit flag

self.exitNow = False

def onKeyboard(self, win, key, scancode, action, mods):

    # print 'keyboard: ', win, key, scancode, action, mods

    if action == glfw.GLFW_PRESS:

        # ESC to quit

        if key == glfw.GLFW_KEY_ESCAPE:

            self.exitNow = True

        else:

            # toggle cut

            self.scene.showCircle = not self.scene.showCircle

def run(self):

    # initializer timer

    glfw.glfwSetTime(0)

    t = 0.0
```

```
while not glfw.glfwWindowShouldClose(self.win) and not  
self.exitNow:
```

```
    # update every x seconds
```

```
    currT = glfw.glfwGetTime()
```

```
    if currT - t > 0.1:
```

```
        # update time
```

```
        t = currT
```

```
        # clear
```

```
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
        # set viewport
```

```
        self.width, self.height =
```

```
            glfw.glfwGetFramebufferSize(self.win)
```

```
        self.aspect = self.width/float(self.height)
```

```
        glViewport(0, 0, self.width, self.height)
```

```
        # build projection matrix
```

```
        pMatrix = glutils.perspective(45.0, self.aspect, 0.1, 100.0)
```

```
        mvMatrix = glutils.lookAt([0.0, 0.0, -2.0], [0.0, 0.0, 0.0],
```

```
                                   [0.0, 1.0, 0.0])
```

```
        # render
```

```
self.scene.render(pMatrix, mvMatrix)

# step

self.scene.step()

glfw.glfwSwapBuffers(self.win)

# poll for and process events

glfw.glfwPollEvents()

# end

glfw.glfwTerminate()

def step(self):

    # step

    self.scene.step()

# main() function

def main():

    print("Starting simpleglfw. "

          "Press any key to toggle cut. Press ESC to quit.")

    rw = RenderWindow()

    rw.run()

# call main

if __name__ == '__main__':
```

main()
